

Transition Motion Tensor: A Data-Driven Approach for Versatile and Controllable Agents in Physically Simulated Environments

Jonathan Hans
Soeseno*
soeseno.jonathan@inventec.com
Inventec Corp.
Taipei, Taiwan

Ying-Sheng Luo*
luo.ying-
sheng@inventec.com
Inventec Corp.
Taipei, Taiwan

Trista Pei-Chun Chen
chen.trista@inventec.com
Inventec Corp.
Taipei, Taiwan

Wei-Chao Chen
chen.wei-
chao@inventec.com
Inventec Corp.
Taipei, Taiwan

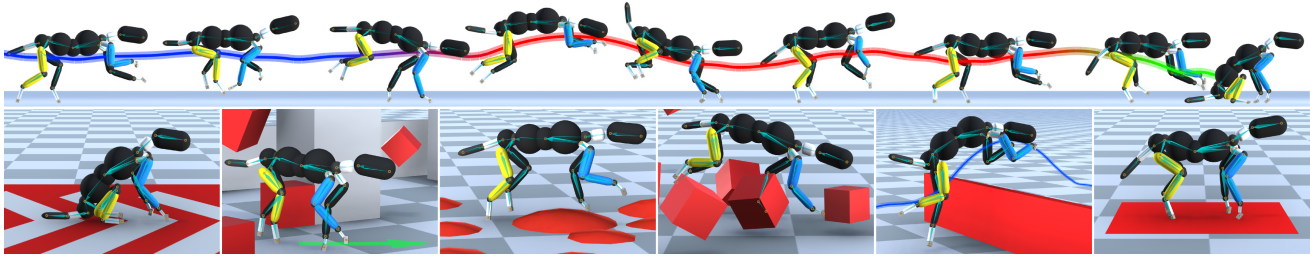


Figure 1: Our data-driven approach compiles individual controllers into a coherent unified controller.

ABSTRACT

This paper proposes the Transition Motion Tensor, a data-driven framework that creates novel and physically accurate transitions outside of the motion dataset. It enables simulated characters to adopt new motion skills efficiently and robustly without modifying existing ones. Given several physically simulated controllers specializing in different motions, the tensor serves as a temporal guideline to transition between them. Through querying the tensor for transitions that best fit user-defined preferences, we can create a unified controller capable of producing novel transitions and solving complex tasks that may require multiple motions to work coherently. We apply our framework on both quadrupeds and bipeds, perform quantitative and qualitative evaluations on transition quality, and demonstrate its capability of tackling complex motion planning problems while following user control directives.

CCS CONCEPTS

• **Computing methodologies** → **Animation**; *Motion Control*.

KEYWORDS

Deep RL, Data-Driven Controller, Novel Transitions

ACM Reference Format:

Jonathan Hans Soeseno, Ying-Sheng Luo, Trista Pei-Chun Chen, and Wei-Chao Chen. 2021. Transition Motion Tensor: A Data-Driven Approach for Versatile and Controllable Agents in Physically Simulated Environments. In

*Joint first authors.



This work is licensed under a Creative Commons Attribution-NonCommercial International 4.0 License.

SA '21 Technical Communications, December 14–17, 2021, Tokyo, Japan

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9073-6/21/12.

<https://doi.org/10.1145/3478512.3488599>

SIGGRAPH Asia 2021 Technical Communications (SA '21 Technical Communications), December 14–17, 2021, Tokyo, Japan. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3478512.3488599>

1 BACKGROUND

The development of character controllers plays a crucial role in movies, games, and robotics. These applications have different requirements for the controller, such as high-fidelity movements that look natural and crisp for movies, responsiveness while maintaining some degree of realism for games, and prioritization of character's safety over motion quality for legged robots. However, controllers for these applications still share similar properties to enable a wide variety of character motions.

To achieve this, one may rely on capturing or authoring an extensive motion dataset to develop a character controller based on the recorded character joint's relations [Holden et al. 2017; Kovar et al. 2002; Starke et al. 2020; Zhang et al. 2018]. While we can enrich the character's motion repertoire by collecting more data, these kinematic approaches require the motion dataset to capture sufficient environment variations to enable character-scene interaction, which could be infeasible to collect for complex interactions.

Their counterpart, the physics-based approaches, can produce plausible scene interactions since they simulate character movements in a physics-enabled environment. Although the constraints from the environment can filter out physically infeasible movements, they also make the controller harder to train. For example, prior works from [Bergamin et al. 2019; Liu et al. 2016; Luo et al. 2020; Park et al. 2019; Peng et al. 2019; Won et al. 2020] produce character controllers that are versatile and controllable but struggle to accommodate a more extensive motion vocabulary because of the requirement to re-train the respective modules when the vocabulary is expanded. Not only the re-training process takes an increasingly longer time with more motions, but it can also alter, or worse yet, break the existing motions.

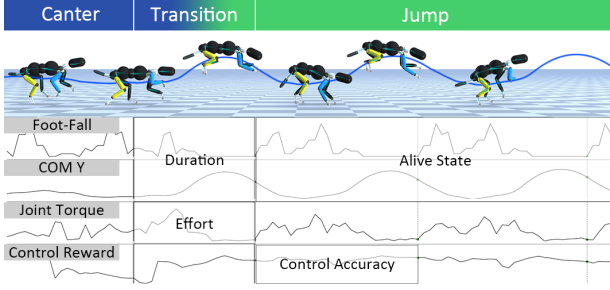


Figure 2: Transitioning from Canter to Jump, the destination controller stabilizes from the switching process, generating a transition of which we record the alive state (η), duration (Δt), effort (e), and control accuracy (α) as the outcome.

To create a data-driven physics-based controller that scales with a large number of motions, we adopt the explicit controller assignment strategy to confine the motion complexity within individual controllers, called template controllers. We then observe that timing is crucial in performing transitions. Therefore, to reliably connect the controllers, we create an external module called Transition Motion Tensor (TMT) that applies to various characters (see Figure 1 and Figure 5). We summarize our contribution as follows,

- A data-driven approach to discover novel and physically accurate transitions outside the motion dataset,
- A scalable framework to link existing controllers of various architectures and training procedures into a coherent, unified controller without additional training, and
- A scheme to integrate user preferences, including the transition effort, duration, and control accuracy, into the controller’s behavior.

2 METHOD

Our goal is to enable a character to perform a wide array of motions in a physically simulated environment. To this end, we train controllable and robust template controllers, followed by unifying them using a data-driven tensor formulated based on user preferences.

2.1 Template Controllers

To achieve a versatile character, we need to model the complex interaction between the desired motions. Instead of using a single controller to learn the entire motion vocabulary, which may require a substantial computation due to entangled motion complexity, we assign individual and physically simulated controllers to imitate specific motions, further referred to as the template controllers. Doing so allows us to confine each motion’s complexity within each template controller, thereby making the training process more tractable and independent. We arrange each motion as a cycle, in which it starts and ends with a similar pose. Inspired by [Luo et al. 2020], we train the template controllers using deep reinforcement learning to achieve life-like character movements, where the controller $\pi(\mathbf{a}_t | \mathbf{s}_t, \mathbf{c}_t)$ yields the action $\mathbf{a}_t$ at each timestep t given the current state of the character $\mathbf{s}_t$ and the control directives $\mathbf{c}_t$.

The template controllers can perform the motions repeatedly in a cyclic manner, where similar character states re-appear every

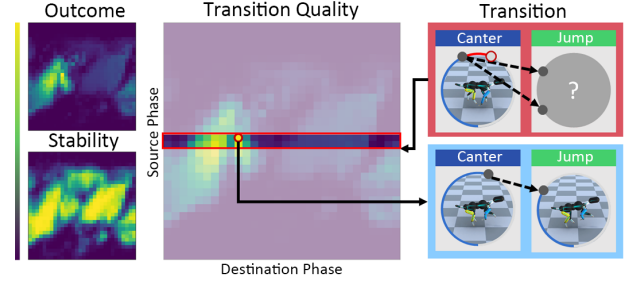


Figure 3: We express the quality of the transition by considering the transition outcome and stability. Querying for high-quality transitions, we can reliably transition between two controllers.

cycle. This condition indicates that the controllers are currently in a *stable state*. However, since the character can interact with the environment, it may stumble due to external perturbations, causing it to break out from the cycle into an *unstable state*. Therefore, to further ensure the robustness of the controllers, we introduce external perturbations and ask the controllers to stabilize the character back towards the cycle. The output is several controllable and robust template controllers $\Psi = \{\pi_1 \dots \pi_6\}$, each specializing on a specific motion in the vocabulary $V = \{\text{Trot}, \text{Canter}, \text{Jump}, \text{Pace}, \text{Stand}, \text{Sit}\}$.

2.2 Transitioning between Controllers

Each template controller only allows the character to perform a specific motion, limiting the character’s ability to solve complex tasks that may require multiple motions to work coherently. For instance, to jump over a wide hole and quickly reach the destination, the character needs to run fast enough, followed by jumping and running towards the goal. However, knowing when to transition between running and jumping is not a trivial task since the character’s state directly affects the transition outcome.

Given that the character is in a particular state with the source controller, as we switch the controller from the source to the destination, the destination controller may have never seen this state. While it tries to recover from this unseen state (i.e., unstable), it consequently generates a new transition motion that neither exists in the source nor the destination controller. That is, the novel transitions are generated by switching between the pair of controllers at the appropriate timing. Without proper consideration of transition timing, we may introduce poses that are difficult for the destination controller to stabilize. For instance, the success of a transition between Canter and Jump relies heavily on the character’s foot touching the ground. Therefore, transitioning from Jump to Canter when the character is mid-air may cause intricacies for the destination controller, leading to a longer time to stabilize, exerting too much effort, deviating from the control objectives, or even worse, causing the character to fall.

To describe the likelihood of successful transitions between source and destination motions, we formulate a 4-D *transition tensor* to record the outcomes of the transitions,

$$\mathcal{T}_{m, \phi, n, \omega} = (\eta, \Delta t, e, \alpha),$$

where $(\eta, \Delta t, e, \alpha)$ denotes the *transition outcomes*. The indices of the tensor include the source $m \in V$ and the destination $n \in V$ motions,

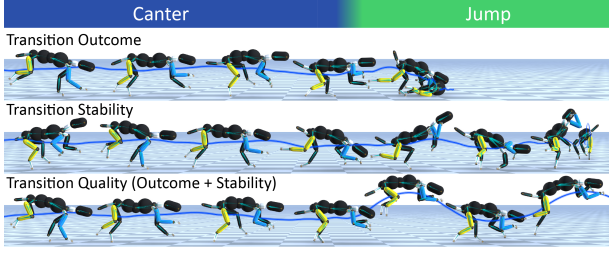


Figure 4: Excluding either the transition outcome or stability, the controller generates sub-optimal transitions.

as well as the source $\phi \in [0, 1)$ and the destination phase $\omega \in [0, 1)$. Note that each component of the tensor is a 4-D vector $(\eta, \Delta t, e, \alpha)$ and should be dependent on $\mathbf{w} = (m, \phi, n, \omega)$, e.g. $\eta \equiv \eta_{\mathbf{w}}$, but we omit them for the sake of brevity. Thus, each element $\mathcal{T}_{\mathbf{w}}$ represents the outcomes of transitions at $\mathbf{w}$ where,

- (η) denotes the *alive state* after the transition, in which $\eta = 1$ means successful transition and $\eta = 0$ if the character falls,
- (Δt) is the *transition duration*, it starts with the switching process and ends after the destination controller stabilizes,
- (e) expresses the *effort* of a transition measured through summation of all joint's torque during the transition, and
- (α) denotes the *control accuracy* measured by the sum of control rewards within the first cycle after the transition.

Figure 2 provides a visual illustration of the transition outcomes.

2.3 Unifying Controllers

With the tensor $\mathcal{T}_{\mathbf{w}}$ describing the likelihood of transitions between a set of template controllers, our goal is to unify all template controllers with transitions that are quick, obey control directives, require low effort, and most importantly, keep the character alive. Therefore, we consolidate the transition outcomes as follows,

$$\Gamma_{\mathbf{w}} = \eta_{\mathbf{w}} \times \frac{\alpha_{\mathbf{w}}}{e_{\mathbf{w}}} \times \exp(-\Delta t_{\mathbf{w}}).$$

The tensor Γ then denotes the consolidated outcome over the indices $\mathbf{w}$. In addition to the outcomes, we also wish to ensure the transition stability where the controller can reliably generate good outcomes with high alive probability. Thus, we define a local neighborhood $\Gamma_{\mathbf{w}}(\delta)$ which is a 2-D sub-tensor of Γ near $\mathbf{w}$, $\mathbf{w} \in \{m, \phi \pm \delta, n, \omega \pm \delta\}$. Then, we can calculate the consistency of the transition outcome $\zeta_{\mathbf{w}}(\delta)$ as the variance of all samples in $\Gamma_{\mathbf{w}}(\delta)$. Similarly, we compute the alive probability of a transition $\eta_{\mathbf{w}}(\delta)$ as the proportion of samples within $\mathcal{T}_{\mathbf{w}}(\delta)$ having $\eta = 1$. The final form of the transition stability then becomes,

$$\psi_{\mathbf{w}}(\delta) = \eta_{\mathbf{w}}(\delta) \times \exp(-\beta \zeta_{\mathbf{w}}(\delta)), \beta = 0.015.$$

Combining the transition stability and the outcomes, the quality of a transition at $\mathbf{w}$ is,

$$\mathcal{Q}_{\mathbf{w}} = \psi_{\mathbf{w}}(\delta) \times \Gamma_{\mathbf{w}}.$$

Next, we look into the importance of each component of the transition quality. Figure 4 demonstrates the effect of removing either the transition outcome or stability. As expected, without considering the transition stability, the controllers generate transitions that cause the character to fall. On the other hand, ignoring the

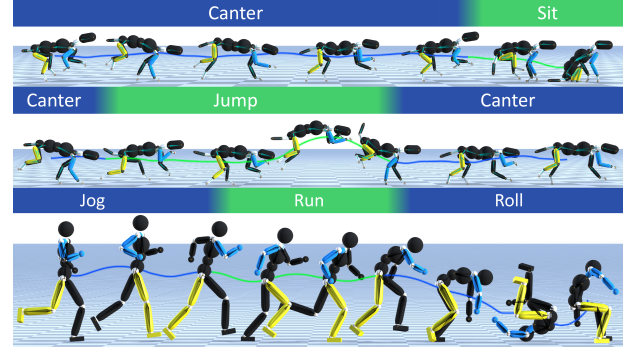


Figure 5: The transition tensor successfully identifies intricate transition strategies for quadruped and biped.

transition outcomes leads to awkward and inefficient transitions. Querying for the highest transition quality based on both components (see Figure 3), we can then reliably perform transitions between two controllers. As a result, applying the same strategy for each pair-wise transition, we successfully unify all template controllers into a single versatile controller that can coherently perform all motions in the vocabulary. More visual illustration is available in the supplementary video.

3 EXPERIMENT

This section evaluates the effectiveness of transition tensor in accommodating various motions in the vocabulary.

3.1 Comparison

We compare our approach **TMT** to three other strategies. First, we consider performing **Motion Imitation**, where we ask a single template controller trained following [Luo et al. 2020] to imitate a reference motion clip containing all desired motions. Alternatively, we can train a **Gating Network** to additively compose the template controllers, as demonstrated by [Won et al. 2020]. Finally, inspired by [Peng et al. 2018], we consider using each template controller's **Value Function** to regulate the transitions between them. We compare these strategies in a setting where we introduce an iteratively growing motion vocabulary using a single workstation equipped with 16 Cores of AMD EPYC 7702, with training speed averaging at 1.6e6 samples/h. Starting with Trot and Canter in the motion vocabulary, we expand it by adding a Jump motion. While the order of motion inclusion may affect the performance, the relative rank of the strategies remains similar.

Table 1 shows the timing results and the success rate of all pair-wise transitions from each strategy when accommodating the motion vocabulary. As expected, strategies that require an additional training process to accommodate the expanded motion vocabulary (Motion Imitation and Gating Network) require a substantial compute time. Even worse, the re-training process may disrupt and alter existing motions. The lower success rate of transitions between Trot and Canter after adding the Jump motion further highlights this problem. In contrast, strategies that do not require any additional training processes preserve the existing motions while achieving better scalability with future vocabulary expansion. However, since the Value Function does not consider the transition outcomes, it

Table 1: The performance of each strategy in accommodating iteratively growing motion vocabulary. Trot, Canter, and Jump motions are denoted as T, C, and J respectively. Our approach achieves a competitive training time and produces the highest success rates for all pair-wise transitions.

Strategy	Vocab.	Train (hours)	Transition (Src/Dst)					
			T/C	T/J	C/T	C/J	J/T	J/C
Motion Imitation	T+C	155.96	0.69		0.94			
	T+C+J	300.00	0.16	0	0.34	0	0.56	0.14
Gating Network	T+C	71.21	1.00		0.83			
	T+C+J	125.26	0.90	0.82	0.57	0.88	0.51	0.74
Value Function	T+C	47.49	0.74		0.38			
	T+C+J	62.03	0.74	0.72	0.38	0.50	0.49	0.45
TMT (Ours)	T+C	52.07	1.00		0.95			
	T+C+J	68.91	1.00	0.96	0.95	0.94	0.96	0.91

generates unstable transitions, which often cause the character to fall. Our approach needs to populate the transition tensor to identify good transitions, which does not update the controller. Therefore, we maintain a competitive compute time compared to the Value Function strategy while offering more reliable transitions highlighted by higher success rates. Figure 6 visually demonstrates how each strategy performs.

3.2 Applicability to Different Characters

Our approach identifies good transition strategies through careful examination of the transition timing and the character’s pose. It applies to various characters, such as 80 DoF quadruped and 34 DoF biped. Moreover, it can also accommodate controllers with different architectures and training procedures, enabling us to use publicly available humanoid controllers, like one developed by [Peng et al. 2018] as our framework’s template controllers. Figure 5 demonstrates the transition tensor’s capability to identify intricate strategies to perform challenging sequences that require precise timing, such as quickly decelerating from cantering to the quadruped’s sitting position and smoothly connecting Jog-Run-Roll for the biped. More results are available in the supplementary video.

4 DISCUSSION AND CONCLUSION

We introduce the transition motion tensor (TMT), a data-driven approach that unifies multiple template controllers into a single coherent controller using novel transitions and careful temporal analysis. The design choice on avoiding the additional training process when including more motions is crucial for scalability, as the process becomes more tractable and independent. Furthermore, by considering the property of the motions, it becomes relatively straightforward to apply our approach for controllers with different network architectures, characters, or even training procedures. This capability is highly desirable as it promotes the reusability of previously designed controllers and shifts the focus of future research directions towards adopting more challenging and intricate motions. In the future, we would like to consider other parameters such as the control and blending parameters, as well as various stylistic properties to further enrich the transition motions.

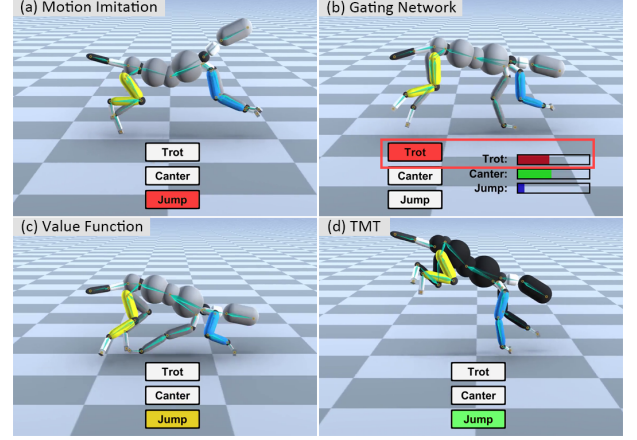


Figure 6: (a) Motion Imitation struggles to learn the Jump motion; (b) Gating Network strategy forgets the existing motion; (c) Value Function generates unstable transitions; and (d) TMT successfully adopts Jump and generates reliable transitions without altering Trot and Canter.

ACKNOWLEDGMENTS

We thank the anonymous reviewers for their insightful comments and our colleagues at Inventec Corporation for their practical feedback that helped improve this paper.

REFERENCES

- Kevin Bergamin, Simon Clavet, Daniel Holden, and James Richard Forbes. 2019. DReCon: Data-Driven Responsive Control of Physics-Based Characters. *ACM Trans. Graph.* 38, 6 (2019), 206.
- Daniel Holden, Taku Komura, and Jun Saito. 2017. Phase-Functioned Neural Networks for Character Control. *ACM Trans. Graph.* 36, 4 (2017), 42.
- Lucas Kovar, Michael Gleicher, and Frédéric Pighin. 2002. Motion Graphs. *ACM Trans. Graph.* 21, 3 (July 2002), 473–482. <https://doi.org/10.1145/566654.566605>
- Libin Liu, Michiel Van De Panne, and Kangkang Yin. 2016. Guided Learning of Control Graphs for Physics-Based Characters. *ACM Trans. Graph.* 35, 3, Article 29 (May 2016), 14 pages. <https://doi.org/10.1145/2893476>
- Ying-Sheng Luo, Jonathan Hans Soeseno, Trista Pei-Chun Chen, and Wei-Chao Chen. 2020. CARL: Controllable Agent with Reinforcement Learning for Quadruped Locomotion. *ACM Trans. Graph.* 39, 4 (2020), 10.
- Soohwan Park, Hoseok Ryu, Seyoung Lee, Sunmin Lee, and Jehee Lee. 2019. Learning Predict-and-Simulate Policies from Unorganized Human Motion Data. *ACM Trans. Graph.* 38, 6, Article 205 (Nov. 2019), 11 pages. <https://doi.org/10.1145/3355089.3356501>
- Xue Bin Peng, Pieter Abbeel, Sergey Levine, and Michiel van de Panne. 2018. Deep-Mimic: Example-Guided Deep Reinforcement Learning of Physics-Based Character Skills. *ACM Trans. Graph.* 37, 4, Article 143 (July 2018), 14 pages. <https://doi.org/10.1145/3197517.3201311>
- Xue Bin Peng, Michael Chang, Grace Zhang, Pieter Abbeel, and Sergey Levine. 2019. MCP: Learning Composable Hierarchical Control with Multiplicative Compositional Policies. In *NeurIPS*.
- Sebastian Starke, Yiwei Zhao, Taku Komura, and Kazi Zaman. 2020. Local Motion Phases for Learning Multi-Contact Character Movements. *ACM Trans. Graph.* 39, 4, Article 54 (July 2020), 14 pages. <https://doi.org/10.1145/3386569.3392450>
- Jungdam Won, Deepak Gopinath, and Jessica Hodgins. 2020. A Scalable Approach to Control Diverse Behaviors for Physically Simulated Characters. *ACM Trans. Graph.* 39, 4, Article 33 (July 2020), 12 pages. <https://doi.org/10.1145/3386569.3392381>
- He Zhang, Sebastian Starke, Taku Komura, and Jun Saito. 2018. Mode-Adaptive Neural Networks for Quadruped Motion Control. *ACM Trans. Graph.* 37, 4, Article 145 (July 2018), 11 pages. <https://doi.org/10.1145/3197517.3201366>